

C200 PROGRAMMING ASSIGNMENT № 2

FUNCTIONS, CONTAINERS, CHOICE

FALL 2024

Dr. M.M. Dalkilic

Computer Science & Data Science

School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

September 20, 2024

In this homework, you'll write functions and use choice. The homework is a little more complex than our last one—as we learn new elements, we can tackle more difficult problems. A few include quick interactive sessions—your are strongly encouraged to do these on your own.

Please start early and push often. **As always, all the work should be with you and your partner; but *both* of you should contribute.** You must complete this before **Friday, September 27, 2024 11:00PM EST**. Please remember that

- You will *not* turn anything in on Canvas.
- You will submit your work by committing your code to your GitHub repository and submitting your code to the autograder. The autograder can be accessed at `c200.luddy.indiana.edu` and you should login via your official IU username and password.
- You do **not manually upload files** to your repository using the GitHub website's "Upload files" tool.
- You must *push* your assignment to GitHub from within VSCode *and* submit your code to the autograder.

If your timestamp is 11:01PM or later, the homework will not be graded. So **do not wait until 10:59 PM** to commit and push your changes. If you have questions or problems with version control, please visit office hours or make a post on Inscribe ahead of time. Since you are working in pairs, your paired partner is shown at the end of this PDF.

Some of these problems were taken or inspired by the excellent introductory *Applied Calculus* by Tan, 2005, *Thinking Mathematically* by Blitzer (5th Ed), *The Concepts and Practice of Mathematical Finance* by Joshi (2nd Ed), and others long out of print.

Instructions for submitting to the Autograder

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).
2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.
3. Given that you already tried points 1-2, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), please try refreshing the page or using a different browser.
4. Once you are done testing your code, please comment out the tests i.e. the code under the `__name__ == "__main__"` section.

Something About Python

We learned a shorthand for conjunction when doing mathematics: if we had two inequalities and they were related in a special way, we could eliminate the explicit `and`. For example,

$$0 \leq x \text{ and } x \leq 10 \quad \leftrightarrow \quad 0 \leq x \leq 10 \quad (1)$$

Python allows this abbreviation too:

```
1 >>> x = 1
2 >>> 0 <= x <= 10
3 True
4 >>> x = -1
5 >>> 0 <= x <= 10
6 False
7 >>> x = 20
8 >>> 0 <= x <= 10
9 False
```

While it's not a great deal shorter, it still is a little easier. Please take advantage of this in the homework when it arises.

Problem 1: Choice

Assume g is a real-valued function defined as:

$$g(x) = \begin{cases} x + 2 & \text{if } x \neq 0 \\ 1 & \text{if } x = 0 \end{cases} \quad (2)$$

For example,

$$g(0) = 1 \quad (3)$$

$$g(1) = 3 \quad (4)$$

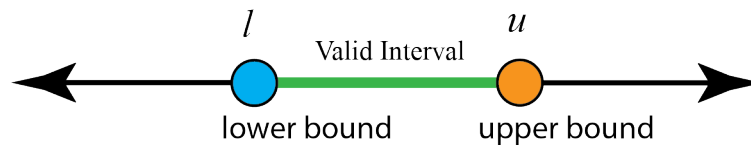
$$g(1.01) = 3.01 \quad (5)$$

Deliverables Problem 1

- Complete the g function.

Problem 2: Senior Citizen Health Care

Often real-world functions requires input to be validated. In this problem, the only valid years are 1977, 1978, ..., 1997. The year, for example, 1976, is not valid. Python allows for a single Boolean expression for intervals. Suppose you're given an interval $[l, u]$ where $l \leq u$. The image below shows valid values (green line and filled circles) and invalid values (black lines).



Let's assume $l = 2$ and $u = 5$ and look at three values: -1.1, 3, 6.

- $2 \leq -1.1 \leq 5 \rightarrow \text{False}$
- $2 \leq 3 \leq 5 \rightarrow \text{True}$
- $2 \leq 6 \leq 5 \rightarrow \text{False}$

Here is a session to confirm this:

```
1 >>> l,u = 2,5
2 >>> x0, x1, x2 = -1.1, 3, 6
3 >>> l <= x0 <= u
4 False
5 >>> l <= x1 <= u
6 True
7 >>> l <= x2 <= u
8 False
9 >>>
```

According to a study of the out-of-pocket cost to senior citizens for health care, $f(t)$ (as percent of income), in year t , where $t = 0$ means 1977, is given by:

$$f(t) = \begin{cases} \frac{2}{7}t + 12 & \text{if } 0 \leq t \leq 7 \\ t + 7 & \text{if } 7 < t \leq 10 \\ \frac{3}{5}t + 11 & \text{if } 10 < t \leq 20 \end{cases} \quad (6)$$

Using a function like this places burden on the user. Let's rewrite the function so that it

- allows using an actual year *e.g.*, 1977
- return a useful error message as a string when the year is incorrect.

The new function definition is:

$$f(t) = \begin{cases} \frac{2}{7}(t - 1977) + 12 & \text{if } 1977 \leq t \leq 1984 \\ (t - 1977) + 7 & \text{if } 1984 < t \leq 1987 \\ \frac{3}{5}(t - 1977) + 11 & \text{if } 1987 < t \leq 1997 \\ \text{"YearError: " + } t & \text{otherwise} \end{cases} \quad (7)$$

As you can observe, the valid years are 1977, 1978, ..., 1997. Any year outside this interval is an error. The “YearError: t ” is an exception that is raised when the wrong year is used.

For example,

$$f(1976) = \text{YearError: 1976}$$
$$f(1977) = 12.0$$
$$f(1985) = 15$$
$$f(1988) = 17.6$$
$$f(2000) = \text{YearError: 2000}$$

Deliverables Problem 2

- Complete the f function.
- We aren't concerned with whether the year is an integer; in other words, it can be a float.
- When $t \notin [1977, 1997]$, then the returned string must be exactly “YearError: $year$ ”, where $year$ is the numerical value that is invalid. There is a single space after the colon.

Problem 3: Cost of OEM parts vs. non-OEM parts

The cost in \$M of OEM parts for years $t = 0, 1, 2$ is given by:

$$h_0(t) = \frac{110}{(1/2)t + 1} \quad (8)$$

The cost in \$M of non OEM parts for the same years is given by:

$$h_1(t) = 26((1/4)t^2 - 1)^2 + 52 \quad (9)$$

The function that describes the difference between the costs for $t = 0, 1, 2$ is:

$$h(t) = \begin{cases} h_0(t) - h_1(t) & t \in [0, 2] \\ \text{"YearError: } t\text{"} & t \notin [0, 2] \end{cases} \quad (10)$$

For example,

$$h(0) = \$32.00 \quad (11)$$

$$h(1) = \$6.71 \quad (12)$$

$$h(1.5) = \$5.88 \quad (13)$$

$$h(2) = \$3.00 \quad (14)$$

$$h(3) = \text{"YearError: 3"} \quad (15)$$

Deliverables Problem 3

- Complete h_0, h_1, h functions.
- Both functions h_0, h_1 are implemented locally.
- Round to two decimal places the return value of h .

Problem 4: Quadratic Formula

The roots of an equation are values that make it zero. For example,

$$x^2 - 1 = 0 \quad (16)$$

$$(x - 1)(x + 1) = 0 \quad (17)$$

Then $x = 1$ or $x = -1$ makes eq. 16 zero. Let's verify this. Taking $x = 1$

$$1^2 - 1 = 1 - 1 = 0 \quad (18)$$

For a quadratic equation (input variable is a power of two), there will be two roots. We'll consider complex numbers later, but for now, we'll assume the roots exist as real numbers. You learned that for a quadratic $ax^2 + bx + c = 0$, two roots x_1, x_2 can be determined:

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad (19)$$

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a} \quad (20)$$

For $x^2 - 1$ the coefficients are $a = 1, b = 0, c = -1$. Then

$$x_1 = \frac{-0 + \sqrt{0^2 - 4(1)(-1)}}{2(1)} \quad (21)$$

$$= \frac{\sqrt{4}}{2} = \frac{2}{2} = 1 \quad (22)$$

$$x_2 = \frac{-0 - \sqrt{0^2 - 4(1)(-1)}}{2(1)} \quad (23)$$

$$= \frac{-\sqrt{4}}{2} = \frac{-2}{2} = -1 \quad (24)$$

We can report the pair of roots as $(1, -1)$ where the left value is the larger of the two. We will assume the three values are given as a tuple (a, b, c) .

$$q((a, b, c)) = \left(\frac{-b + \sqrt{b^2 - 4ac}}{2a}, \frac{-b - \sqrt{b^2 - 4ac}}{2a} \right) \quad (25)$$

For example, we'll use $x^2 - 1$, $6x^2 - x - 35$, and $x^2 - 7x - 7$:

$$q((1, 0, -1)) = (1.0, -1.0) \quad (26)$$

$$q((6, -1, -35)) = (2.5, -2.3333333333333335) \quad (27)$$

$$q((1, -7, -7)) = (7.887482193696061, -0.8874821936960613) \quad (28)$$

Deliverables Problem 4

- Complete the `q()` function.
- The pair of real-valued roots *must* have the larger one as the first value of the tuple.
- You can not use the `cmath` module for this problem or any other module.

Problem 5: AI Grading System

AI is now all over the news—disrupting almost all industries. In this problem, you’ll be writing an AI system to determine whether a human has answered a simple mathematical problem correctly. Your goal is to replace instructors. Students are given an expression for example: $5 \times 5 =$ and then write an answer. corresponding answer for example: 4. The function determines if the answer is the correct output of the given operation or not. There are four operations: multiplication, addition, subtraction, and division. The data is in the form of a list:

$$[arg_1, op, arg_2, answer]$$

$arg_1, arg_2, answer$ are floats and op is the string “*”, “**”, “+”, “-”, “/”. For example, $[1, “*”, 2, 3]$ means $1 * 2 = 3$ and is False. On the other hand, $[2, “**”, 3, 8]$ means $2^3 = 8$ and is True.

The function takes the data list and returns Booleans True or False. We do not do any error checking for this problem.

$$eq([arg_1, op, arg_2, answer]) = \begin{cases} \text{True} & \text{if } arg_1 \text{ op } arg_2 = answer \\ \text{False} & \text{otherwise} \end{cases} \quad (29)$$

Here are some examples:

$$eq([14, “/”, 2, 7]) = \text{True} \quad (30)$$

$$eq([20, “*”, 19, 381]) = \text{False} \quad (31)$$

$$eq([20, “*”, 19, 380]) = \text{True} \quad (32)$$

$$eq([2, “**”, 3, 8]) = \text{True} \quad (33)$$

$$eq([1.1, “-”, 1, .1]) = \text{False} \quad (34)$$

What’s happening for the last one (34)? Remember, we have representation issues on a computer:

```
1 >>> 1.1 - 1
2 0.10000000000000009
```

We’ll learn to deal with this as we progress through the semester.

Deliverables Problem 5

- Complete the eq function.
- You do not have to account for the zero division error. We won’t check for divide by zero case for division operation.

Problem 6: COVID Symptom Presentation

Research from USC suggests that some COVID-19 symptoms appear in a particular order:

- A. fever
- B. cough and muscle pain
- C. nausea or vomiting

Let's assume we want to write a program to indicate the likelihood of someone having COVID-19 based on the order in which they present symptoms A, B, and C. How many different orders are there? There are $3! = 6$ possible orders (leftmost being first): ABC, CBA, CAB, BAC, BCA, ACB. The task is to determine the likelihood of COVID-19 infection given a list of any of these symptoms. For this initial problem, we'll sort the symptoms by hand, which only works for small data sets. For larger sets, we would need a quantifiable way to sort. My reasoning is that A is the most important, B is second, and C is third, but I'd have to confirm this **computational approach** biologically. For now, let's just use the following:

very likely	likely	somewhat likely
ABC, ACB	BAC, BCA	CAB, CBA

The function `covid(symptoms)` is given a string `symptoms` ABC,ACB,...,CBA. Return the string "very likely" if the pattern as 'A' as the first symptom, "likely" for the strings that have 'B' as the first symptom, and "somewhat likely" for the rest. There are many different ways to implement this. Here is a run:

```
1 print(covid('ABC'), covid('ACB'))
2 print(covid('BAC'), covid('BCA'))
3 print(covid('CAB'), covid('CBA'))
```

with output:

```
1 very likely very likely
2 likely likely
3 somewhat likely somewhat likely
```

Deliverables Problem 6

- Complete the function `covid`.
- For the output strings-note that there is one whitespace between "very" and "likely" and one whitespace between "somewhat" and "likely".

Problem 7: Maximum

Write a function `max2d` that takes two numbers and returns the larger. Using only `max2d` write a function `max3d` that takes three numbers and returns the largest.

$$\text{max2d}(x, y) = \begin{cases} x & \text{if } x > y \\ y & \text{otherwise} \end{cases} \quad (35)$$

$$\text{max3d}(x, y, z) = \text{max2d}(x, \text{max2d}(y, z)) \quad (36)$$

For example,

$$\text{max3d}(1, 2, 3) = 3 \quad (37)$$

$$\text{max3d}(1, 3, 2) = 3 \quad (38)$$

$$\text{max3d}(3, 2, 1) = 3 \quad (39)$$

Deliverables Problem 7

- Complete both `max2d`, `max3d` functions.
- You cannot use Python `max`.
- You can only use `if` statements.
- `max3d` can only use `max2d`.
- Create your own tests

Interestingly, you can use arithmetic and Boolean values for `max2d` (you cannot use it for this homework)

```
1 def m(x, y):
2     return (x > y)*x + (x <= y)*y
3
4 print(m(1, 2))
5 print(m(2, 1))
```

Has output:

```
1 2
2 2
```

Problem 8: Voting

To complete this problem, we'll remind you of the `len()` function for containers and strings, but also `sum()`. The `len()` function takes a list, tuple, or string and returns the size. If the containers are empty, then zero is returned. The `sum()` function (which we'll implement when we learn about looping), takes either a list or tuple, and gives the sum when the members are either integers, floats, or complex numbers. We'll not consider complex numbers for this problem. For example:

```
1 >>> x = [1,0,1]
2 >>> sum(x)
3 2
4 >>> sum(x + x)
5 4
6 >>> sum((3,2,4))
7 9
8 >>> sum([1,0,1,1,0])
9 3
```

You'll implement the function `decision(data)` that takes a list `[name0, name1, votes]` where `name0`, `name1` are strings for the two candidates and `votes` is a list of zeros and ones. A zero is a vote for `name0`; a 1 is a vote for `name1`. The function will return a tuple: (name, winning number of votes, total votes cast). In case of tie, the format is slightly different, below is a run of three examples:

```
1 data0 = ['B', 'Z', [0,1,1,0,1,0,0]]
2 print(decision(data0))
3 data1 = ['B', 'Z', [1,0,1]]
4 print(decision(data1))
5 data2 = ['B', 'Z', [1,0,1,0,1,1,0,0]]
6 print(decision(data2))
```

produces

```
1 ('B', 4, 7)
2 ('Z', 2, 3)
3 ('tie', 4, 4)
```

The first data, `data0`, has 'B' receiving 4 votes (zeros) and 'Z' receiving 3 votes (ones); therefore, 'B' wins with 4 votes out of 7. Similarly for the second case. In case, both candidates got the same number of votes (as shown in the third example), the output is a 'tie' and the number of votes that each candidate received.

Deliverables Problem 8

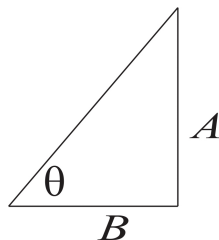
- Complete the `decision(data)` function. The function takes a list of three elements: `[name0, name1, votes]`. the `votes` variable is a list of 0,1. You can assume it is *never* empty.
- You should use `len()` `sum()` but no other built-in functions.

Problem 9: Tan Chat

While we've seen None used in errors, its Boolean value is useful:

```
1 >>> not None
2 True
3 >>> bool(None)
4 False
5 >>> None and True
6 >>> bool(None and True)
7 False
8 >>>
```

Remember, when not coerced into a type, None doesn't return anything. This problem asks you to implement a "smart" problem solver for triangles.



To remind you, in a right triangle:

$$\tan \theta = \frac{A}{B} \quad (40)$$

$$\tan^{-1}(\tan \theta) = \tan^{-1}\left(\frac{A}{B}\right) \quad (41)$$

$$\theta = \tan^{-1}\left(\frac{A}{B}\right) \quad (42)$$

The equations on lines (41-42) show the inverse tangent. The math module has both tangent (as `math.tan`) and inverse tangent (`math.atan`). These use radians instead of degrees, so you'll have to convert. The math module also has functions to convert: `math.radians()` takes degrees and returns radians, `math.degrees()` takes radians and returns degrees. To remind you:

$$\pi \text{ radians} = 180 \text{ degrees}$$

In this problem, you'll write a "smart" problem solver. The user calls the function `solve(theta, opposite, adjacent)`. When one of the variables is None, the function returns the answer for that missing value. Remember, None is considered False as a Boolean value. When all the variables have values, the function returns whether the equation is True or False. Here's a run:

```
1 print(solve(5, None, 105600))
2 print(solve(None, 9238.9, 105600))
3 print(solve(5, 9238.8, None))
4 print(solve(5, 9238.8, 105600))
5 print(solve(5, 9100, 105600))
```

has output:

```
1 9238.802868337576
2 5.000052300754411
3 105599.96721475148
4 True
5 False
```

Let's see the math. For the first call, we have this where $\theta = 5^\circ$, $opposite = None$, $adjacent = 105600$:

$$\tan(5^\circ) = \frac{None}{105600} \quad (43)$$

$$\tan(5^\circ) \times 105600 \rightarrow 9238.802868337576 \quad (44)$$

The function should return $\approx 9238.802868337576$. Please note that on your system, the exact value may differ starting 9th or 10th place after the decimal. This is because our systems differ in hardware and software configuration due to which some difference can be obtained in large floating numbers. This won't be a problem when you submit to the Autograder. Here's the Python in an interactive session to confirm this:

```
1 >>> import math
2 >>> math.tan(math.radians(5))*105600
3 9238.802868337576
```

Here's another example when $\theta = None$, $opposite = 9238.9$, $adjacent = 105600$

$$\tan(None) = \frac{opposite}{adjacent} \quad (45)$$

$$None = \tan^{-1}\left(\frac{9238.9}{105600}\right) \rightarrow 5.0000523007 \quad (46)$$

Here's the Python to confirm this:

```
1 >>> math.degrees(math.atan(9238.8/105600))
2 4.999998455537281
3 >>> math.degrees(math.atan(9238.9/105600))
4 5.000052300754411
```

Observe that one decimal value difference affects the angle slightly. The hardest challenge is to determine when the equation is True because of representation. Suppose someone has this call:

```
1 print(solve(5,9100,105600))
```

Here are the values according to Python:

```
1 >>> math.tan(math.radians(5)), 9238.8/105600
2 (0.08748866352592401, 0.08748863636363635)
```

These **should** be considered to be equal, but they are not. We can use the math function `isclose()` to help us. The function takes two values and an optional `abs_tol = tolerance`, where tolerance is a real number value, then returns `True` if the values up to the tolerance value specified in the `abs_tol` argument are the same. Where do the two values differ?

```
1 >>> x,y = math.tan(math.radians(5)), (9238.8/105600)
2 >>> x - y
3 2.7162287655202455e-08
4 >>> math.isclose(x,y,abs_tol=0.001)
5 True
6 >>> math.isclose(x,y,abs_tol=10e-9)
7 False
8 >>> math.isclose(x,y,abs_tol=10e-8)
9 True
```

We can see that if the tolerance is 0.001, we're sure to affirm these values are equal.

Deliverables Problem 9

- Complete the solve function.
- *hint*: you can use the variable's value of `None` to determine what you solve for!.
- You must use the math module including `isclose()`.
- Use an absolute tolerance of 0.001 when implementing the `isclose()` function. Please read on your own if you want to know more about the function.

Problem 10: Purchasing a Home without PMI

We saw the formula for an annuity with interest r compounded yearly n over t years with a payment of P :

$$A = \frac{P((1 + \frac{r}{n})^{nt} - 1)}{(\frac{r}{n})} \quad (47)$$

This formula can be used to find what your payment needs to be to finance, say, a house. When purchasing a house, if you're unable to pay at least 20% of the price, you're required to pay an additional 0.2% to 3.0% (Private Mortgage Insurance) of the original loan. PMI is a heavy burden. You want to write a program that, given a home's price H , allows you to determine what is the minimum down payment to avoid PMI. For this problem we will assume you want to have the down payment in two years $t = 2$ and you'll make monthly payments $n = 12$. For example, a home costing \$250,000 requires $.2 \times \$250,000 = \$50,000$ downpayment due at signing. Note that downpayment is found by calculating 20% of the total price. Assuming you're getting 6% compounded monthly for two years, then you'll need to pay \$1,966.03 per month. Here is a run:

```
1 #problem 10
2     home_price, rate = 250000, 6/100
3     t,n = 2,12                                #years, monthly
4     payment = future(home_price,rate)
5
6     print(f"{n} payments yearly for {t} years requires ${payment}")
7
8     #confirm this achieves 50000
9     A = round(payment*((1 + rate/n)**(t*n)-1)/(rate/n),2)
10    print(A)
```

producing

```
1 12 payments yearly for 2 years requires $1966.03
2 49999.99
```

Line 9 uses Eq. 47. As not unexpected, there's a tiny roundoff problem. A penny won't make a difference.

Deliverables Problem 10

- To implement the function, you'll need to solve for P in Eq. 47.
- For this problem, we assume $n = 12$ and $t = 2$, *i.e.*, monthly payments for two years. Therefore, the formal parameters for the problem are home price and interest rate.
- The function `future(A,r)` returns the payment rounded to two decimal places.

Problem 11: Queries as Linear Inequalities

A predicate is a Boolean valued function. The function takes input and returns either true or false. Mathematical lines can function as Boolean functions. Indeed, a model might have one or more linear equations that describe a solution set. For example, consider this linear equation in $\mathbb{R}$ (Euclidean space):

$$3x - 4y > 12 \quad (48)$$

The predicate describes where the most profit is obtained or where an NPC is or return signals from LiDAR. The Fig. 1 is shaded for Eq. 48. There are three regions: blue indicating where Eq. 48 is false, the line (red) where it is neither false nor true, and the unshaded region where it is true.

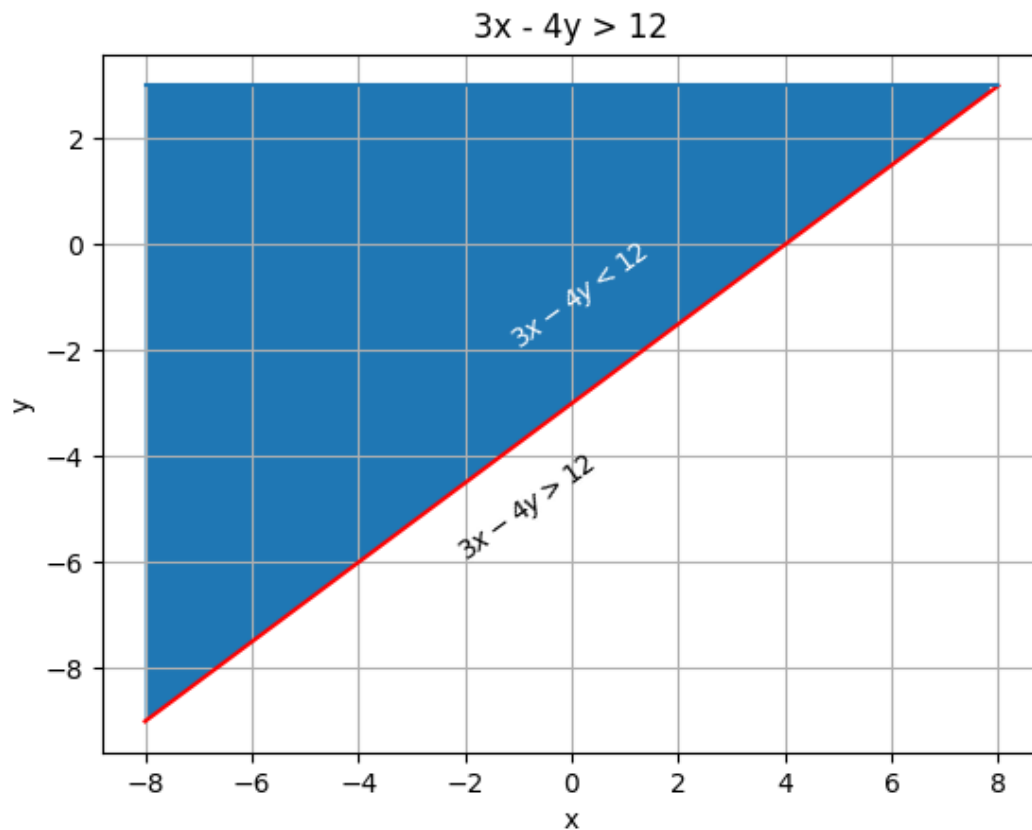


Figure 1: Visualizing a linear inequality.

Given a point (x, y) , we can determine the Boolean value. Let's look at two queries: point 1 = $(-6, 0)$ and point 2 = $(4, -5)$. In Fig. 2 we've plotted both points:

Point 1 lies above the line. What Boolean value does it give?

$$3(-6) - 4(0) = -18 + 0 = -18 > 12 \rightarrow \text{False} \quad (49)$$

This makes sense. Let's look at the other point:

$$3(4) - 4(-5) = 12 + 20 = 32 > 12 \rightarrow \text{True} \quad (50)$$

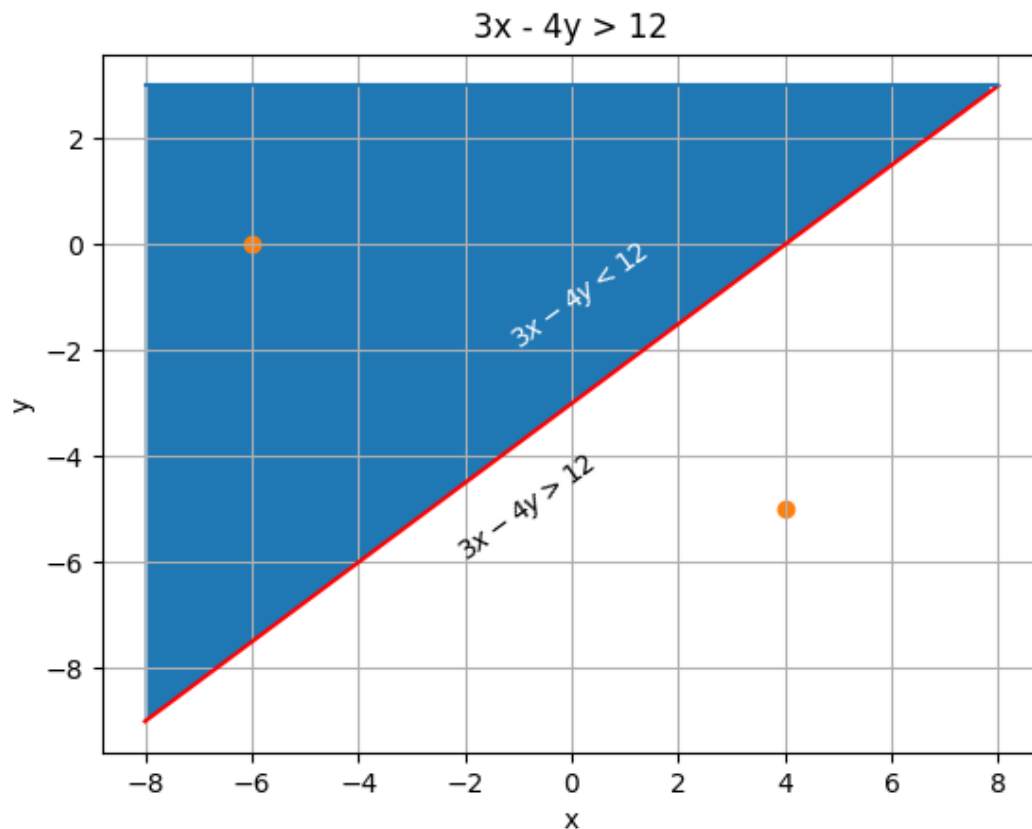


Figure 2: Visualizing a linear inequality.

Now we can implement a function to determine the truth-value of a point.

```

1 def linear_query(a,b,c,point):
2     x,y = point
3     pass
4
5 if __name__ == "__main__":
6     point1 = (-6,0)
7     point2 = (4,-5)
8     print(linear_query(3,-4,12,point1))
9     print(linear_query(3,-4,12,point2))

```

produces:

```

1 False
2 True

```

Your task is to implement this simple function. Assume that we always have this inequality:

$$ax + by > c \quad (51)$$

Your function will take these parameters and a point and return True or False.

Deliverables Problem 11

- Implement the function. Assume the point on a line returns false.

Problem 12: Two Trains Diverged on Rusty Tracks and Sorry I could not Take Both...

Here is a popular problem. Two trains are leaving from a station traveling west. Train 1 leaves at 12:00P traveling 40mph. Train 2 leaves two hours later at 60mph. When will Train 2 reach Train 1? The trains will have traveled the same distance d for the time that both travelled t given the speeds of Train 1, s_1 and Train 2, s_2 and that Train 1 has already traveled $t_0 \times s_1$:

$$ts_2 = ts_1 + t_0s_1 \quad (52)$$

$$t(s_2 - s_1) = t_0s_1 \quad (53)$$

$$t = \frac{t_0s_1}{s_2 - s_1} \quad (54)$$

$$= \frac{2(40)}{60 - 40} = \frac{80}{20} = 4 \quad (55)$$

So the time will take 4 hr. A couple of remarks—it's better to not use numbers explicitly when studying a problem. They get in the way and have a way of often taking on wrong values. Secondly, Eq. 54 provides a *general* solution to every train problem of this ilk. As a data scientist/computer scientist we can take advantage of that and write a function that'll compute *any* answer. Let's see if we can generalize even further. Two trains are leaving their respective stations, one directly east at 40mph and the other directly west at 60mph. Each train is traveling to the other station. The distance between the two stations is 400 m. When will they meet? Try to work this out before looking at the answer.

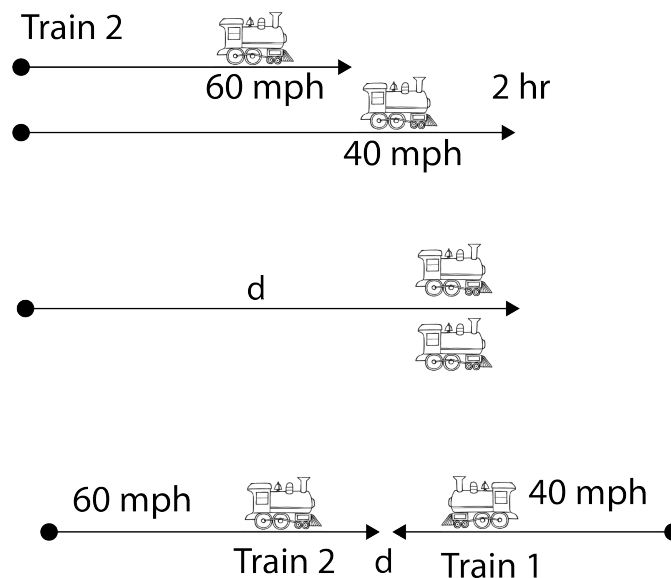


Figure 3: Visualizing two common train problems.

$$ts_1 + ts_2 = d \quad (56)$$

$$t(s_1 + s_2) = d \quad (57)$$

$$t = \frac{d}{s_1 + s_2} \quad (58)$$

$$t = \frac{400}{40 + 60} = 4 \quad (59)$$

We can write a function for this too.

Deliverables Problem 12

- Implement the two functions `train_type1` and `train_type2` that describe the two problems.
- Round time to two decimal places.

Problem 13: Using the math module

Given a set of things $A = \{x_1, x_2, \dots, x_n\}$ suppose we want $k \leq n$ where order doesn't matter. For example, from a group of either people, you need to pick three for taking notes. How many distinct groups are there?

Combinatorics gives us an answer:

$$C(n, k) = \frac{n!}{(n-k)!k!} \quad (60)$$

For our problem:

$$C(8, 3) = \frac{8!}{(8-3)!3!} \quad (61)$$

$$= \frac{40320}{120 \cdot 6} = 56 \quad (62)$$

The math module (as we've discussed in lecture) has factorial. We can confirm this:

```
1 >>> import math
2 >>> math.factorial(8)
3 40320
4 >>> math.factorial(8-3)
5 120
6 >>> math.factorial(3)
7 6
8 >>> math.factorial(8)/(math.factorial(8-3)*math.factorial(3))
9 56.0
```

All we need to do now is implement this function. The following code:

```
1 def C(n,k):
2     pass
3
4 if __name__ == "__main__":
5
6     n,k = 8,3
7     print(C(n,k))
```

has the following output:

```
1 56.0
```

Deliverables Problem 13

- Implement the function using the math module.
- Since this never returns fractions so you can leave it as it is, i.e. because there's division, so return a float.

Problem 14: Reusing Code

For this last problem, you'll reuse code in a previous problem, *i.e.*, the solver for tangent problems. Here is a general problem: For an equilateral triangle whose side is x length, determine the area of the largest circle that can be inscribed.

Please take a look at Fig. 4. We can take advantage of the geometry of both shapes. As you can observe we can use the tangent function to determine r . In fact, we can use our solver.

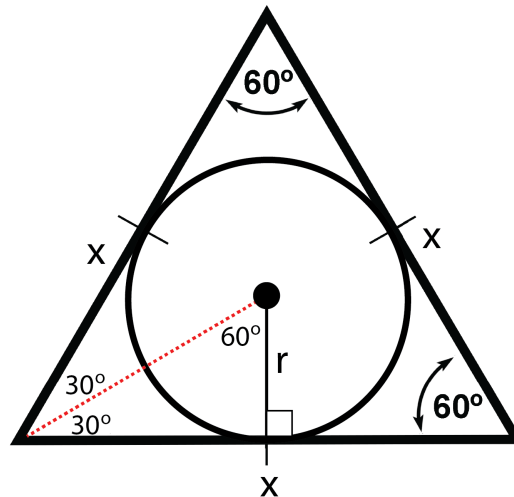


Figure 4: The largest circle inscribed in an equilateral triangle with side length x .

The following code:

```
1 def circle(x):  
2     #uses solve  
3     pass  
4  
5 if __name__ == "__main__":  
6  
7     x = 14*math.sqrt(3)  
8     print(circle(x))
```

has output:

```
1 153.94
```

Deliverables Problem 14

- Implement the function using the solve function you've created.
- Round the answer to two decimal places.

Problem 15: Space-Time

Using Einstein's theory of relativity (actually two theories) we believe that as someone travels closer to the speed of light, time dilates. We can write an equation to describe this time contraction phenomenon as:

$$time_contract(t_{earth}, v) = t_{earth} \left(1 - \left(\frac{v}{c}\right)^2\right)^{\frac{1}{2}} \quad (63)$$

where t_{earth} is the time that has elapsed on earth, c is the speed of light ($186000 \frac{mi}{s}$), v is the velocity we're traveling. For example, if we're traveling 90% of light speed, then one year on earth will seem only four months in space. If 75 years has elapsed on earth, then in space we've experienced about than 32.7 years. We add the following variables to the function so we can focus only on time:

```
1 def time_contract(earth_time):
2     c = 186000 #speed of light in mi/sec
3     v = .9 * c
4     pass
5
6 earth_time = [1,25,50,75] #years that elapsed
7
8 for et in earth_time:
9     print(time_contract(et))
```

produces:

```
1 0.44
2 10.9
3 21.79
4 32.69
```

Deliverables Problem 15

- Implement the `time_contract()` function.
- Round the answer to two decimal places.

Problem 16: System of Solutions

For this problem we assume a solution exists. A linear equation of two variables is:

$$ax + by + c = 0 \quad (64)$$

It is used ubiquitously in all domains. Given two linear equations of two variables,

$$a_1x + b_1y + c_1 = 0 \quad (65)$$

$$a_2x + b_2y + c_2 = 0 \quad (66)$$

implement two functions: `linear_solver(eq1,eq2)` returns a tuple of the solution (x,y) and `confirm(eq1,eq2,solution)` that returns true if the solution satisfies the system. The following code:

```
1 data = [((1,1,-5),(3,-1,-3)), ((-1,-1,5),(2,-1,-4))]  
2  
3 for d in data:  
4     print(linear_solver(*d))  
5     print(confirm(*d, linear_solver(*d)))
```

produces:

```
1 (2.0, 3.0)  
2 True  
3 (3.0, 2.0)  
4 True
```

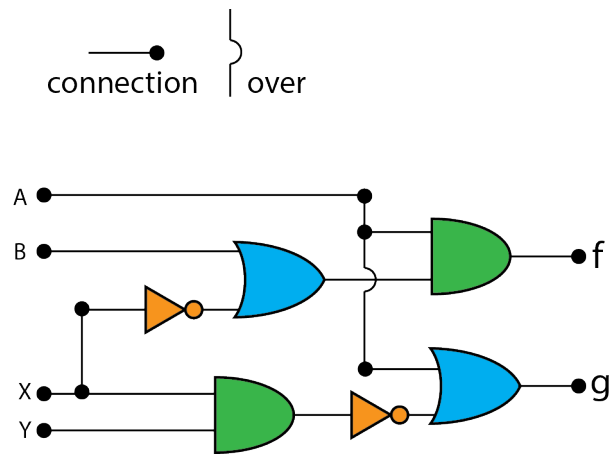
Observe we're sending two tuples of coefficients to the first function and the coefficients along with the solution as a tuple to the second.

Deliverables Problem 16

- Implement the `linear_solver()` and `confirm()` functions.
- We assume that the data will include only systems that have a solution.
- We cannot explicitly use any modules *etc.*

Problem 17: Implementing a Circuit

It's a curious cycle: we develop hardware to run software to design hardware: really to improve the system. Implement the following circuit in Python:



The dots represent connecting signals. The half-circle means the wire is above the wire below. Return the tuple cast as integer. Observe that we're returning a tuple of two values (f,g). The following code:

```
1 input = [0,1]
2 for A in input:
3     for B in input:
4         for X in input:
5             for Y in input:
6                 print(f"{A} {B} {X} {Y} {circuit(A,B,X,Y)}")
```

produces:

```
1 0 0 0 0 (0, 1)
2 0 0 0 1 (0, 1)
3 0 0 1 0 (0, 1)
4 0 0 1 1 (0, 0)
5 0 1 0 0 (0, 1)
6 0 1 0 1 (0, 1)
7 0 1 1 0 (0, 1)
8 0 1 1 1 (0, 0)
9 1 0 0 0 (1, 1)
10 1 0 0 1 (1, 1)
11 1 0 1 0 (0, 1)
12 1 0 1 1 (0, 1)
13 1 1 0 0 (1, 1)
14 1 1 0 1 (1, 1)
15 1 1 1 0 (1, 1)
16 1 1 1 1 (1, 1)
```

Deliverables Problem 17

- Implement the `circuit()` function.
- Make sure to cast the Boolean `f` and `g` as integers.

Problem 18: To Err is Human, but OSHA Keeps Count Anyway

The Occupational Safety and Health Administration (OSHA) aims to make work safe. Suppose you're writing some software to help determine the number of accidents in a manufacturing complex that has three units a,b,c. Each month the unit lists the number of accidents. Implement the function `def accident_ordering(c1,c2,c3)` that takes three tuples (a,alst), (b,blst), (c,clst) of the unit name with a tuple of integers corresponding the number of accidents (see examples below). The function will return the list of units, in ascending order, of number of accidents; in other words, list the units ordering by the fewest accidents. The following code:

```
1 data = [(( 'a' , (1,2)), ( 'b' , (1,3)), ( 'c' , (1,4))),
2         (( 'a' , (1,2)), ( 'b' , (1,4)), ( 'c' , (1,3))),
3         (( 'a' , (1,3)), ( 'b' , (1,2)), ( 'c' , (1,4))),
4         (( 'a' , (1,4)), ( 'b' , (1,2)), ( 'c' , (1,3))),
5         (( 'a' , (1,3)), ( 'b' , (1,4)), ( 'c' , (1,2))),
6         (( 'a' , (1,4)), ( 'b' , (1,3)), ( 'c' , (1,2)))]
7
8 for d in data:
9     print(accident_ordering(*d))
```

produces:

```
1 ( 'a' , 'b' , 'c' )
2 ( 'a' , 'c' , 'b' )
3 ( 'b' , 'a' , 'c' )
4 ( 'b' , 'c' , 'a' )
5 ( 'c' , 'a' , 'b' )
6 ( 'c' , 'b' , 'a' )
```

Deliverables Problem 18

- Implement the `accident_ordering()` function.
- You will use `sum()`, but no other functions.
- You can only have a single `return` in the function.
- Assume each list has at least one number.

Programming partners

bthenson@iu.edu, ssanjee@iu.edu

adhimat@iu.edu, ydpatil@iu.edu

hdjour@iu.edu, aralover@iu.edu

karbern@iu.edu, hshyama@iu.edu
evbutts@iu.edu, mtay@iu.edu
jbriar@iu.edu, kokulski@iu.edu
bboyapat@iu.edu, angzamor@iu.edu
tycarmo@iu.edu, kenmoses@iu.edu
edeitchl@iu.edu, jawegman@iu.edu
mrb9@iu.edu, anangla@iu.edu
iyersan@iu.edu, anrajput@iu.edu
xmdrew@iu.edu, wkpierce@iu.edu
kbussel@iu.edu, bobbyun@iu.edu
goldfusl@iu.edu, sandro@iu.edu
jergalan@iu.edu, isvoor@iu.edu
keswar@iu.edu, jlaytin@iu.edu
maboncos@iu.edu, vpottete@iu.edu
evarcarm@iu.edu, bsati@iu.edu
heissler@iu.edu, cammlanc@iu.edu
ldeveau@iu.edu, rnalluri@iu.edu
skapure@iu.edu, zhangsuo@iu.edu
rdotsu@iu.edu, mclindbe@iu.edu
alehe@iu.edu, hsh1@iu.edu
zahaidar@iu.edu, lvumpa@iu.edu
cjo3@iu.edu, aidpride@iu.edu
sergonza@iu.edu, sahasrir@iu.edu
jigarnes@iu.edu, tkthang@iu.edu
mitja@iu.edu, whitedyl@iu.edu
tborder@iu.edu, ayorgen@iu.edu
aabdulw@iu.edu, tlichten@iu.edu
adagar@iu.edu, timolaws@iu.edu
meiyhe@iu.edu, prmundra@iu.edu
felkhatt@iu.edu, youngct@iu.edu
okirchen@iu.edu, maymat@iu.edu
jaeblank@iu.edu, rosanghv@iu.edu
mahchaud@iu.edu, amarynow@iu.edu
dh43@iu.edu, jo16@iu.edu
aare@iu.edu, bawsung@iu.edu
augguy@iu.edu, wl52@iu.edu
clickc@iu.edu, cmellgre@iu.edu
jamcham@iu.edu, jhl14@iu.edu
mugabr@iu.edu, jw363@iu.edu
gkreutes@iu.edu, jacloise@iu.edu
armstan@iu.edu, olsoncs@iu.edu

ahsans@iu.edu, rvasude@iu.edu
aarepal@iu.edu, audscha@iu.edu
cmfalk@iu.edu, joevu@iu.edu
agrief@iu.edu, thebmill@iu.edu
salvchri@iu.edu, mt87@iu.edu
shkami@iu.edu, sumuth@iu.edu
andedaws@iu.edu, jakpatel@iu.edu
ethdeatr@iu.edu, prnemani@iu.edu
adekolaw@iu.edu, aml4@iu.edu
mdevara@iu.edu, tnerkar@iu.edu
bronburk@iu.edu, alzemlya@iu.edu
adatwani@iu.edu, shamonei@iu.edu
ldeel@iu.edu, nikhmorg@iu.edu
keswa@iu.edu, isapittm@iu.edu
svdatla@iu.edu, swaschow@iu.edu
am280@iu.edu, missavin@iu.edu
jackchap@iu.edu, kpazdur@iu.edu
cedrchen@iu.edu, kzerbino@iu.edu
mbenites@iu.edu, jwubelho@iu.edu
grhamlin@iu.edu, atsarver@iu.edu
florech@iu.edu, kw125@iu.edu
bskendal@iu.edu, djtheodo@iu.edu
ahmealsh@iu.edu, smungee@iu.edu
jk314@iu.edu, natkwon@iu.edu
tbrose@iu.edu, cjlomax@iu.edu
mgoenka@iu.edu, roldenbu@iu.edu
eabilius@iu.edu, vivitran@iu.edu
nicdelg@iu.edu, ijrussel@iu.edu
byersjac@iu.edu, conaitis@iu.edu
enkarl@iu.edu, kr15@iu.edu
ckbarts@iu.edu, mmaplet@iu.edu
bowenbm@iu.edu, cmolaski@iu.edu
rkoston@iu.edu, rysalina@iu.edu
emhjames@iu.edu, pp22@iu.edu
coobratt@iu.edu, jiajtang@iu.edu
zalsamri@iu.edu, gsackie@iu.edu
agotanco@iu.edu, dematt@iu.edu
rgyasini@iu.edu, anryoun@iu.edu
michdool@iu.edu, dommonte@iu.edu
crigonza@iu.edu, smitfi@iu.edu
elehaged@iu.edu, jakepars@iu.edu

alneyadk@iu.edu, jacdstev@iu.edu
abeathea@iu.edu, navann@iu.edu
nbriere@iu.edu, smitaa@iu.edu
jejea@iu.edu, nmeely@iu.edu
efoltyn@iu.edu, parishar@iu.edu
twchase@iu.edu, kyalane@iu.edu
jkuk@iu.edu, rpolu@iu.edu
kevgross@iu.edu, jopantoj@iu.edu
nwconn@iu.edu, lemarc@iu.edu
dheer@iu.edu, reshetty@iu.edu
bahls@iu.edu, cnshimi@iu.edu
nantinao@iu.edu, sasubb@iu.edu
ejaini@iu.edu, auskirvi@iu.edu
ajkouts@iu.edu, gl25@iu.edu
colguth@iu.edu, neenpate@iu.edu
shkani@iu.edu, marcmarq@iu.edu
natkling@iu.edu, cmekat@iu.edu
lafly@iu.edu, westje@iu.edu
abrking@iu.edu, sxanders@iu.edu
owflynn@iu.edu, mmalviya@iu.edu
doandre@iu.edu, drimawi@iu.edu
rycostin@iu.edu, cjrodenb@iu.edu
langarbe@iu.edu, mmmatias@iu.edu
dakomi@iu.edu, braxluns@iu.edu
gibsontn@iu.edu, kiloscot@iu.edu
heoji@iu.edu, bgtruong@iu.edu
elscheng@iu.edu, hmahapat@iu.edu, mzuhair@iu.edu
skyangel@iu.edu, negia@iu.edu
cdasari@iu.edu, mloko@iu.edu
akoduru@iu.edu, camoverm@iu.edu
chjflor@iu.edu, ansjmeht@iu.edu
chfurlow@iu.edu, mitcal@iu.edu
iblay@iu.edu, aamuse@iu.edu
garmenda@iu.edu, ndmyer@iu.edu
ssdatla@iu.edu, liaobrie@iu.edu
sydforem@iu.edu, zs15@iu.edu
dayinla@iu.edu, dyllopez@iu.edu
ojallow@iu.edu, slopezve@iu.edu
dgiffor@iu.edu, ssp4@iu.edu
srkagema@iu.edu, mjseymou@iu.edu
nimjain@iu.edu, antonegr@iu.edu

mjfarrin@iu.edu, manpate@iu.edu
ebederma@iu.edu, kroussel@iu.edu
mahichan@iu.edu, hailreid@iu.edu
ramgowda@iu.edu, mszach@iu.edu
talfares@iu.edu, nieywill@iu.edu
alurs@iu.edu, martibr@iu.edu
jb108@iu.edu, seuryu@iu.edu
adj18@iu.edu, tunnitha@iu.edu
adodaro@iu.edu, ksabloak@iu.edu
btbasing@iu.edu, bogawa@iu.edu
drmiguth@iu.edu, adenyu@iu.edu
mailic@iu.edu, pmiesch@iu.edu
egilmour@iu.edu, fsafianu@iu.edu
asgurram@iu.edu, ishyadav@iu.edu
blbinder@iu.edu, jquansah@iu.edu
zkabacin@iu.edu, irodeman@iu.edu
nganesh@iu.edu, isschetty@iu.edu
jdkakare@iu.edu, robreese@iu.edu
ethhanna@iu.edu, lisalaza@iu.edu
pikhatri@iu.edu, adivaidy@iu.edu
concraft@iu.edu, jvmenach@iu.edu
rugaddam@iu.edu, nealsing@iu.edu
rogoenka@iu.edu, sholove@iu.edu
reclay@iu.edu, rvenigal@iu.edu
jagoodal@iu.edu, rheashah@iu.edu
dajuhnke@iu.edu, froy@iu.edu
clarkina@iu.edu, jasreese@iu.edu
ghargro@iu.edu, davepais@iu.edu
idasilva@iu.edu, fasrasoo@iu.edu
dkembert@iu.edu, aishravi@iu.edu
sdondeti@iu.edu, cardnorr@iu.edu
fcohan@iu.edu, pjminne@iu.edu
damaatki@iu.edu, vpotluri@iu.edu
sabyrap@iu.edu, clushell@iu.edu
blfost@iu.edu, evanmayo@iu.edu
rpkeatin@iu.edu, jl367@iu.edu
pranchan@iu.edu, cs155@iu.edu
btcunnin@iu.edu, mahonm@iu.edu
hayburge@iu.edu, mkreddy@iu.edu
ag46@iu.edu, nshnoble@iu.edu
kadlakha@iu.edu, ncw1@iu.edu

jkittur@iu.edu, jrwaren@iu.edu
boydmad@iu.edu, dslipch@iu.edu
rbernfel@iu.edu, msharsh@iu.edu
coubenne@iu.edu, isundara@iu.edu
bedani@iu.edu, sstickl@iu.edu
nfarzane@iu.edu, adimaran@iu.edu
bdgonzal@iu.edu, jjrinald@iu.edu
vkabra@iu.edu, zanmille@iu.edu
ag38@iu.edu, aposton@iu.edu
kkunshet@iu.edu, everma@iu.edu
tbashee@iu.edu, mawlorch@iu.edu
addfishe@iu.edu, rasahu@iu.edu
jahjacks@iu.edu, risvphil@iu.edu
ljhorman@iu.edu, tersacks@iu.edu
jbosio@iu.edu, dmw6@iu.edu
brjduck@iu.edu, omwells@iu.edu
blstcoop@iu.edu, prschulz@iu.edu
eynkansa@iu.edu, antzamor@iu.edu
mfbah@iu.edu, aluckhau@iu.edu
sanjais@iu.edu, abbywebe@iu.edu
dqchoe@iu.edu, soyedele@iu.edu
agahlau@iu.edu, lydiwang@iu.edu
lddial@iu.edu, kevwong@iu.edu
owencape@iu.edu, jewach@iu.edu
mabana@iu.edu, jskains@iu.edu
mkulekci@iu.edu, ainlukas@iu.edu
mthinman@iu.edu, nguyen17@iu.edu
drussow@iu.edu, jmsequei@iu.edu
vicbarre@iu.edu, muksingh@iu.edu
nifredri@iu.edu, dmonroyh@iu.edu
jjithesh@iu.edu, klohiya@iu.edu
skalluru@iu.edu, tanilitt@iu.edu
ehchoo@iu.edu, wschrama@iu.edu
ttfender@iu.edu, roberjuv@iu.edu
etferba@iu.edu, corawang@iu.edu
smhamrah@iu.edu, zz67@iu.edu
abondada@iu.edu, ibshaw@iu.edu
pchitwoo@iu.edu, kalovert@iu.edu
nafike@iu.edu, elwesley@iu.edu
nikdevra@iu.edu, matyun@iu.edu
cgcheane@iu.edu, emasseng@iu.edu

mgbekele@iu.edu, rvarrier@iu.edu
bcarpine@iu.edu, lukthiel@iu.edu
laigrant@iu.edu, andyni@iu.edu
radutta@iu.edu, olesmith@iu.edu
yc127@iu.edu, aramanu@iu.edu
adguhan@iu.edu, mjo4@iu.edu
benchart@iu.edu, nmenne@iu.edu
bbeer@iu.edu, ctsvetan@iu.edu
ygajjar@iu.edu, ew38@iu.edu
eberens@iu.edu, dajshaw@iu.edu
joglickm@iu.edu, sunathan@iu.edu
shkakkar@iu.edu, alejmora@iu.edu
dbarbari@iu.edu, vpremku@iu.edu
ealfalah@iu.edu, anupand@iu.edu
ndodoh@iu.edu, esspatel@iu.edu
yaljabri@iu.edu, stewrach@iu.edu
ajafowl@iu.edu, athason@iu.edu
xbchrist@iu.edu, cnorcutt@iu.edu
banksda@iu.edu, shwanara@iu.edu
leidaugh@iu.edu, taylbryo@iu.edu
jaccryst@iu.edu, jnimmala@iu.edu
herncr@iu.edu, ozopich@iu.edu
rkoranne@iu.edu, cpmann@iu.edu
lajith@iu.edu, cpmathew@iu.edu
carthugh@iu.edu, kalerobi@iu.edu
akeshav@iu.edu, alniehau@iu.edu
bgelard@iu.edu, samyadav@iu.edu
nitiarun@iu.edu, marsh1@iu.edu
saskenaz@iu.edu, gsskippe@iu.edu
moazimi@iu.edu, ozsachs@iu.edu
tgebrew@iu.edu, aidsteph@iu.edu
vbhattad@iu.edu, vnathany@iu.edu